BioSpecInfo

Test Documentation — BioSpecInfo

Samuele Pio Provenzano

Versione **bsi-v203** · commit `1a58c5a`

Generato il 2026-10-14

Documento generato automaticamente dal sorgente Markdown versionato nel repository. La versione e il commit stampati in ogni piè di pagina identificano esattamente lo stato del software descritto.

Test Documentation — BioSpecInfo

Field	Value
Software	BioSpecInfo
Version described	`bsi-v203`
Purpose	Describe how the tests are organised, how to run them, what they cover and where they leave gaps.

1. Kinds of test present

Kind	Present	Notes
End-to-end (E2E)	✅	Playwright on headless Chromium against a local HTTP server. It is the prevailing form.
Data verification	✅	Scientific data compared against an independent source
Stability tests	✅	Long sessions, exhausted storage, degraded network
Documentation/code consistency	✅	Verifies that what the documentation promises really exists
Isolated unit tests	❌	See §6
Instrumented code coverage	✅	`audit_copertura`, with Chromium's profiler: no build, no source rewritten. See §6

Why E2E and not unit tests

The application is made of classic scripts operating directly on the DOM, with no modules and no dependency injection: there are no units that could be isolated without a deep rewrite. The defects that actually occurred — a blurred canvas, a button unreachable because its container is collapsed, an AI provider that stops answering — **would not have surfaced** from a unit test anyway: they live in the integration between code, DOM and browser.

The choice is therefore reasoned, not endured. Its limits remain, and are declared in §6.

2. How to run the tests

Requirements

```
npm install                      # playwright-core only
python3 -m http.server 8899 &    # RDKit WASM requires an HTTP context
```

> Without a server every test fails: opening the files over `file://` prevents the WebAssembly modules from loading.

The whole battery, with a report

```
node tools/genera-evidenza.js
```

It runs every bench, transcribes its full output and produces `docs/evidence/RAPPORTO-VERIFICA.md` with environment, commit and SHA-256 digests. It exits with code `0` only if no bench fails.

```
node tools/genera-evidenza.js --veloce   # skips the benches that need a browser
```

The `--veloce` option is useful for a quick check but **is not sufficient for an attestation**: the benches it skips are precisely the ones measuring real behaviour.

A single bench

```
node tools/verifica-farmaci.js           # a stand-alone tool
node tools/verifica-farmaci.js --json    # machine-readable output

cd tools/banchi && node test_spettri.js  # one E2E bench
```

The E2E benches live in **`tools/banchi/`**, versioned like the rest of the repository; the `BSI_BANCHI` variable allows another folder to be pointed at.

> **It has not always been so, and that is worth saying.** Up to version `bsi-v168` the default folder was the working area of the session in which the benches had been written: **32 of the 39 benches were not in the repository**. Whoever cloned it and ran the documented command found seven, and received no warning — because a missing bench was counted but did not make the battery fail, which closed with "CONFORMANT" and zero failures.
>
> Both things have been corrected: the benches are versioned, and **a missing bench makes the outcome NON-CONFORMANT**. A bench that is not there is not a bench that passed.

---

3. Composition of the battery

65 benches, grouped by what they demonstrate.

3.1 Scientific data

Bench	Verifies	Checks
`audit_farmaci`	Structure ⟷ weight, duplicates, and the **formula rebuilt from the graph** against ChEMBL's for every entry declaring a provenance	263 entries, 36 with a source
`test_spettri`	Functional-group recognition on reference molecules	36
`test_spettri_ui`	Grafting of the engine into the app's spectrum-drawing points	16
`test_assi`	Axis conventions in the SVG spectra	13
`test_assi_canvas`	Axis conventions in the canvas spectra	6
`test_costanti`	Physical-constant lookup and refusal of ambiguities	45
`audit_dati`	Consistency of the tabulated data	29
`test_farm_ui`	That the Drug Atlas **draws** every drug it holds in memory, category by category, and that a **never-labelled** category still appears	11
`test_astro`	The astrochemistry data: molecular weight ⟷ formula for the 30 interstellar molecules against IUPAC atomic weights, no repeated celestial body, every quiz question with a valid answer among its options	13
`test_datasci`	The maths the Data Science section **displays**: R^2 on an exact relation, standardised weights in the true ratio, ROC on hand-computable values, behaviour on pure noise, and that **no section is left unreachable**	23
`test_simmetria`	Point groups, normal modes, IR/Raman selection rules	26
`test_cheminfo`	The cheminformatics engine: Tanimoto and Dice against hand-computable values, Morgan and MACCS fingerprints, Butina clustering, Bemis–Murcko scaffolds, PCA, kernel ridge, scaffold split, **null model by label scrambling**, grouped cross-validation, CSV and SDF exports, **contrast of the nine panels**, agreement between lab and engine, fragmentation, **matched pairs against known effects**, substructure search, SAR table, model comparison in both directions, enrichment (EF and BEDROC) on hand-computable cases, conformal intervals and their **refusal on classification**	189
`test_spettrolettore`	The spectrum reader and the models that follow from it: ASDF symbol by symbol, explicit and DIF/DUP-compressed JCAMP, prominence peaks on **built** spectra **and on pure noise**, curve versus peak list, neutral losses on toluene, ESOL against four experimental values, four drug-likeness filters reported separately, random forest both ways, **a single entry point** (one button, empty `accept`, routing checked on PDF/`.docx`/`.txt`/`.jdx`/`.csv`/a disguised binary/an image) and the version badge compared with `BSI_APP_VERSION`. **And the relationship between the two IR tables**: `SPEC_DATA.ir` is for studying, `BANDE_IR` for assigning, they are not a duplicate — but the assignment umbrella must **cover** the reference for each of the 23 groups present in both, and not be arbitrarily wider (margin derived from the data, not invented)	66

Bench	Verifies	Checks
`test_nmr`	Per-atom NMR prediction: two separate molecule sets (tuning and **validation**, the latter never used to choose parameters), the signal count on molecules whose count is known, refusals on an unreadable SMILES and on an empty string, the **history** with duplicate detection by InChI key, and the **peak ↔ structure ↔ 3D** sync verified on the indices, not on how many atoms light up	64
`test_nmr2d`	Correlation maps counted on the graph: HSQC with inverted sign on CH_2, COSY between non-equivalent protons, HMBC two and three bonds away. **In both directions**: hexafluorobenzene gives no HSQC spots, benzene **no** off-diagonal COSY spots, methane **no** HMBC spots despite having four protons	40
`test_digitalizza`	A spectrum extracted from a **figure**, over seven constructed images whose Gaussian centres are known to the cm^{-1}: bare figure, with frame and labels, with a grey grid, with a black grid, with frame **and** grid, dashed, in %T. **Mean 2.2 cm^{-1}** deviation, **worst 7.3** — one column is worth ~4 cm^{-1}, which is the limit of the figure and not of the method. **In both directions**: from a blank sheet and from the app icon **no** curve comes out; the crude fallback on the same framed figure finds **one** peak instead of three; on a logarithmic axis the midpoint is the geometric mean and on a linear axis the arithmetic one. %T→A conversion on exact values (50 %T → A = 0.301) and a **full round trip** of the exported JCAMP, read back by the reader itself	45
`test_gioco`	The round language switcher and the game. The switcher: round, top right, **at the measured title height** (the two boxes overlap), fourteen flags on two shells, the disc inside the screen on a phone, and **both ways** the language actually changing and coming back identical. The game: **19 question generators each tried** (one that produces nothing does not "run"), 600 sound questions and **none ambiguous** on ranges — with the guard that those generators were not switched off to make the check pass; **60 levels** (12 × 5 difficulties) with no gap wider than the jump and no step higher, every boss with its shooting element; and the game actually played, from the title to **victory over the final boss**. Plus: the loop that **stops** when the section closes (zero frames), nothing overflowing at 390 px, controls ≥ 44 px with accessible names. **The title screen and the pause menu, measured in the running app on a phone-sized window, portrait and landscape**: the name drawn on the canvas (pixels counted) and animated (two different frames), and **not repeated in HTML**; the pause menu with the guide, fourteen flags and the settings; the language switch actually switching; the run frozen while paused; and **every menu item reachable by thumb** — inside the screen with nothing on top of it: this is the check that found the navigation bar covering «resume»	105
`audit_statico`	The classes of bug this project has already had, searched across **all** files instead of where they showed up: `calc()` without spaces (the browser drops the rule), secrets in logs, API keys in the code, plaintext passwords, `target="_blank"` without `noopener`, duplicate ids in the markup, unbalanced scripts. The report **flags and does not block**, because those classes have false positives. Three measures **do** block, and have a written baseline: verbatim duplicated code blocks (**0**), redefined global functions (**11**, the file's declared "additive patch" style), of which with identical bodies (**0**). If they grow, the battery fails	3 gates

Bench	Verifies	Checks
`test_articoli`	That no function has two entry points, and that the articles are not hand-written. Ten tabs inside a single Spectra centre, with each identifier present in the document **exactly once**; no repeated id, no duplicate button, no section reachable by two roads; Spectra out of the menu and with its dedicated button. Then the articles: the links tested as **pure functions** of the identifier, the Nobel explanations matched to the topic of the motivation and **none** invented when it is not recognised, and the decisive proof — an **injected** response with a name that exists nowhere, which must appear on screen. **Three distinct failures**: failed request, empty answer, unrecognised shape	58
`test_geom3d`	The 3D geometry built from the graph, measured against literature values: bonds, valence angles, alkyne linearity. **In both directions**: benzene comes out planar and cyclohexane does **not**. And the proof that makes the peak↔atom link legitimate: every heavy atom has **the same index and element** as the predictor's	29
`test_documento`	Opening any document and working through the problem inside: PDF, Word and text genuinely opened and the content verified; the data recognised **with their provenance**. **In both directions**: an image declares there is no optical character recognition and invents no working, an unknown format is refused rather than guessed, $CDCl_3$ does not become the compound's formula, without an MS label there are no masses	47

A bench that verifies a model must also verify it when the model is supposed to fail. `test_cheminfo` exercises the null model in both directions: on a learnable signal the true R^2 (0.394) beats all eight label-scrambled twins, with a margin of 0.287; on the **same molecules with random labels** the R^2 (-0.199) does **not** beat them. A check that verifies only the first case would pass even if the guard were wired to "true".

A "contrast 0" that did not cover the new surface. `verifica-accessibilita` measures what is visible, and the six panels of the Cheminformatics section stay hidden until the analysis has been run: it had never seen them. `test_cheminfo` now runs the analysis, opens the panels in turn and measures contrast over **460 text elements**, with the WCAG formulas rewritten inside the bench — asking the code under test how good its own contrast is, is not measuring. The bench also demands that more than 250 were measured: if the panels failed to open, the zero would be empty and the check would fail.

Three peaks found prove nothing on their own. A detector that is too sensitive finds thirty where there are three; one that is too strict finds none. Neither case raises an error. `test_spettrolettore` starts from three Gaussians **built** at 1715, 2950 and 3400 cm^{-1} under noise and demands that the reader find three — 1716, 2952, 3402 — and then, on the **same noise without the Gaussians**, that it find **zero**. Without the second check the first would pass even with a detector that marked every wiggle. The same holds for the random forest: out-of-bag R^2 0.90 on the real relationship and **−0.16 on pure noise**.

> **The defect was in my counter, not in the data.** The first run of `verifica_farmaci_v187` returned **0 formulas out of 36**: every molecule came out as `C<n>`, all carbons and no hydrogens. RDKit's `get_json` format is **commonchem**, where an atom carries only the fields that **differ** from the defaults — `z` is 6 and `impHs` is 0 — so that an atom written `{}` is a carbon. I was reading `a.element`, which does not exist in that format. Thirty-six data points that look wrong all at once are almost always a wrong counter.

3.2 AI agent

Bench	Condition simulated
`test_404`	The provider has retired the model
`test_503`	Temporary overload
`test_ko` · `browser_ko`	Provider unreachable from the browser
`test_firma`	Opaque reasoning signatures to be returned
`test_tetto`	Token ceiling learned from the error
`test_attesa` · `test_attesalunga`	Timeouts and imposed waits
`test_doppioinvio`	Two overlapping submissions
`browser_prova`	Reachability measurement without a key
`test_nucleo` · `browser_prov`	Agentic loop and provider selection
`caccia_ai`	Malformed answers, truncated stream, 502 with HTML, Stop during the answer, panel closed halfway, hostile text

3.3 Stability

Bench	What it puts to the test
`audit_stabilita`	96 sections opened 5 times, full storage, corrupted data, bursts of clicks
`audit_promesse`	Rejected and unhandled promises, with a healthy network and a dead one
`audit_quota`	`localStorage.setItem` forced to fail on 10 pages
`test_sw`	Offline, degraded network, update while working
`test_filemanager`	Store unavailable, space exhausted, failed writes
`test_visore3d`	WebGL context consumption while browsing molecules

3.4 Interface

`browser_reset` · `browser_proxy` · `browser_proxyui` · `browser_rdkit` · `browser_lab` · `browser_frontiera` · `test_aggiorna` · `test_guidaproxy` · `test_fluidita` · `test_lingue` · `test_mol3d` · `test_spettrolettore`

`test_fluidita` **— 15 checks.** It measures how long the page stays *blocked* at every section switch, across all 91. It found three defects no other bench could see: the "3D PRO Viewer" froze the page for **797 ms** on the first click — the profiler attributes them to 3Dmol's first `render()`, which compiles the WebGL context's shaders; the **296 figures** in "Synthesis" (5,351 SVG nodes) were all drawn before anything was shown, while three are visible on opening; and **296 timers 40 ms apart** were scheduled when that section opened — eleven and a half seconds of wake-ups, each calling `getTotalLength()` on every stroke of every figure, including those off screen.

After the fixes: median switch **16 ms**, worst **87 ms**, **no section above 100 ms** (there were two, the worst at 1,166 ms).

At version `bsi-v188` the same bench caught two things. First: with 263 drugs instead of 233, «Pharmacology» blocked the page for **159 ms** and the count of sections over 100 ms went back to **three** — the battery had passed by a hair the time before, and a limit exceeded intermittently protects nothing. The cards are now drawn in 8 ms slices: **55 ms**, zero sections over 100.

Second concerns the bench itself. The viewer threshold was **80 ms absolute**; with the same published code, on different containers, the same click measured 14, 22, 98, 105 and 192 ms, because WebGL here is **SwiftShader** and the cost is shader compilation. Such a threshold measures the machine: it fails on identical code and would pass on worsened code. Even a **ratio** against a calibration taken in the same run does not hold — by the time the bench reaches the viewer WebGL is already warm (2.9 ms) while the viewer pays the cold cost: the ratio swung between 14× and 36×. The claim became **structural**, which is also the original defect: immediately after the click handler returns the WebGL context **must not exist**, shortly after it **must**. Measured 0 and 1; and the measurement is not vacuous, because on a second click — when the canvas is already there — it returns 1.

`test_lingue` **— 64 checks.** It watches two new features and two different ways of lying without noticing.

The language. A switch that "translates the application" is easy to write and hard to keep honest: someone changes an Italian label and its translation is no longer found. The defect is invisible — the label stays Italian in the middle of English — unless someone counts. The bench counts: **160 skeleton elements, 160 translated, none left**. It also demands the two opposite proofs: that switching to English **actually changes** the text (an empty dictionary would pass "everything translated" without moving a letter) and that switching back to Italian restores it **character for character**.

The molecular languages. Conversions are checked against external facts — seven literature InChI keys **written inside the bench** — and the full SMILES → molfile → SMILES round-trip is required. Formulas are counted by hand.

The IUPAC namer. 26 names written by hand in two languages, and seven molecules that **must be refused**: a namer that never refuses is a namer that invents.

Tested in both directions: restoring the `trim()` that breaks molfiles and removing one translation makes the bench fail three checks out of twenty.

The doors. Since v184 the bench also checks that the two features are **reachable**: a navigation button, a section that really draws, and — a check born from a real defect — that neither shows the "🚧 Section under construction" placeholder that the empty-section fallback stamps 800 ms after a click into any section left empty. It also checks that the ✨ menu entry **navigates** to the section rather than opening an overlay window, and that no duplicate id exists with both sections drawn.

The six languages. For each one two things are demanded together: full skeleton coverage (**162 of 162**) and that the text **really changes** — an empty dictionary would pass "everything translated" without moving a letter.

`test_mol3d` **— 46 checks.** It watches the molecule-formation animation and the bond-angle measurement, two additions with two different ways of looking right without being right.

The animation. "It is there" or "it is not" cannot be seen by inspecting the code: a loop that spins and redraws the same thing passes any check on the function's existence. The bench looks at the **pixels**: at 130 ms the canvas must be nearly empty — dust — and at 2.3 s there must be a molecule. Measured: **532 lit pixels → 2,334**. If the two numbers were equal there would be no animation, however the program is written. It also checks that the loop **stops** when stopped: a removed canvas with the loop still running is a drain paid in battery.

The angles. A number with a "°" next to it looks like a measurement. They are checked on hand-built geometries whose value is known **by construction**, not taken from a table: regular tetrahedron **109.4712°**, water 104.47°, CO_2 180°, BF_3 120°, ammonia 106.13°. Tolerance 0.05°.

The refusal. On a **flat** structure (z = 0) the angles button must be disabled and the reason written: on 2D coordinates a bond angle is not a bond angle, and showing it would be the worse of the two errors because it looks like data. Tested both ways: a structure with $z \neq 0$ must not be mistaken for flat, or no angle would ever be shown.

The first draft of this bench read the control bar **after** stopping the loop, which removes it: it found zero buttons and then declared "each has an accessible name" over an empty set. A check that passes because it has nothing to look at is exactly the defect this project chases; the check now demands two buttons **and** two names.

The selection. The viewer chooses the angle by clicking atoms: one shows all the angles at that atom, two the bond length, three the angle with the vertex at the second click, four the **dihedral**. The bench checks all four cases on ethanol — C–C 1.509 Å, C–C–O 111.9°, H–C–C–O 60.7° — and demands that two **unbonded** atoms be declared as such rather than passed off as a bond: a distance is not a bond.

The dihedrals. Checked on four hand-built conformations: eclipsed 0°, gauche 60°, orthogonal 90°, anti 180°. Here too the hand-written table corrected an error of mine, not of the program: with the coordinates I had chosen the dihedral is 120°, not 60°, because looking along the central bond the projected vector sits at (−0.5, 0.866).

The arrival order. "Skeleton first, hydrogens after" is a **measurable** property, not an intention: the bench reads each atom's progress halfway through the formation and requires every heavy atom to be ahead of every hydrogen. That check found a real defect: the elastic

function that produces the bounce is not monotonic, and using it as the progress too meant an atom that started earlier could look "less arrived" than one that started later. The two quantities are now separate — the progress is monotonic, the elastic factor only does the bounce — and the bounce, previously clipped away, is visible.

Half the checks are the *opposite proofs*, because a page that builds nothing is instantaneous: the WebGL canvas must appear anyway, all 296 figures must exist shortly after, no frame may stay empty, printing must not come out mute, and the search must filter even cards whose figure is not yet born. And the bench counts the sections it traversed: on ten instead of 89 the timings would be excellent and the measurement worthless.

3.5 Security and accessibility

Bench	Verifies
`verifica-sicurezza`	API keys in tracked files, clear-text passwords, secrets in `wrangler.toml`, telemetry, scripts from external domains
`audit_storia`	That no credential has EVER entered the repository: **1,551 distinct versions of text files** across **390 commits**, with 9 patterns. It refuses to pass on a shallow clone
`verifica-accessibilita`	WCAG AA contrast, accessible names, field labels, alternative text, heading hierarchy, `lang` attribute — across 13 pages
`audit_mobile`	That at **390 px** the page does not scroll horizontally, across all 96 sections — **and** that no section clips content into unreachability: an element wider than its section, with no scrolling ancestor, is text that is present and cannot be read. A wide table inside a scrolling container is not reported
`audit_rete`	That **no user data leaves the device**: a canary value seeded into 71 stores, the app used across 6 pages and 96 sections, the URL, headers and body of every request inspected
`audit_copertura`	How many bytes of JavaScript are **actually executed** while walking the application: 49.89 %, recorded and defended

`audit_storia` — 6 checks. `verifica-sicurezza` examines *tracked* files, i.e. the present state: it answers "there is no key in the repository today", not the question that matters for a public repository. A key added in one commit and removed in the next passes `verifica-sicurezza` forever and stays readable to anyone who clones, because git does not forget. If that happens, the remedy is not a fix-up commit: it is revoking the key.

This bench examines every *version* of every text file ever committed. Result: **zero credentials across 1,551 versions and 390 commits**.

Two defences against itself, because here a perfect number can come from less surface or from a blind pattern:

- **The clone must not be shallow.** A `--depth 1` holds a fraction of the

history: the bench would run, find nothing, and claim "no credential anywhere in history" having seen a tenth of it. If it finds `.git/shallow` it fails and names the command to run.

- **The patterns are tested before being trusted.** A wrong pattern finds

nothing and is indistinguishable from a clean repository: each of the 9 must match a crafted example and must *reject* a near-miss, and both are tested inside a tail of noise containing digits, as a real file would have.

The second defence was born from a defect in this very bench. The AWS pattern required a digit via `(?=.*\d)` — always satisfied in a large file, because the digit lies further ahead: it found 53 matches, all the same string `AKIAAAAAAAAAAAAAAAAA`, sixteen "A"s, the zero region of a base64 image. The near-miss did not catch it because it was too short to contain digits. An AWS identifier is now judged by the *variety* of its sixteen characters, and the near-misses carry their tail.

Tested in the other direction too: with a local commit containing a well-formed AWS key the bench fails naming the blob; once the commit is removed, it passes again.

> **The obstacle belonged to the tool, not to the problem.** For three versions code coverage was declared unmeasurable, with this reason: "instrumenting would require introducing a build, which the application does not have". That was true of `c8` and `istanbul`, which rewrite the source before running it. But **Chromium collects it by itself**, inside the engine, without touching a byte of the file served: `Profiler.startPreciseCoverage`, which Playwright exposes as `page.coverage.startJSCoverage()`. No build was needed: what was needed was noticing that the constraint belonged to the tools chosen.
>
> **The first result was 100 % on every file**, including 3.6 MB of `index.html`. That was the bench measuring nothing, in its most insidious form: a perfect number. V8 emits, for each function, an outer range with the number of entries, and inside it ranges with `count: 0` for what was not executed. Summing the positive ranges sums the whole file. One does the opposite: start from the total and subtract the union of the empty ranges.
>
> The real number is **49.89 %**, and it holds the same pact as the accessibility debt: recorded, and the bench fails if it falls. An absolute threshold ("80 %") would be an invented figure; "it must not get worse" is a promise that can be kept.

Why a canary and not a list of requests. SEC-02 — "no personal data leaves the device" — was verified by checking the two *mechanisms* of exit: empty telemetry variable, no script from an external domain. That is real evidence but indirect: it says "I cannot see how it would leave", which is not "it did not leave". Looking instead at the list of hosts contacted would be just as weak, because it concludes from absence of evidence.

The canary reverses the burden. An unrepeatable string is written into the user's own data — notes, chats, API keys, settings, progress — then the application is **used**, and every outgoing request is opened and read. If the string appears nowhere, it is because it really did not leave.

```
BSI_PROVA_FUGA=1 node tools/banchi/audit_rete.js   # must FAIL
```

The bench carries its own proof of working: with that variable it causes a leak on purpose, towards a **declared** host — contacting it is legitimate, sending it the user's data is not. If it does not fail then, the instrument is the thing that is broken.

Why a bench on horizontal overflow. UI-03 says "the app must work at 390 px", and the matrix gave it as verified by `audit_stabilita`, which in a phone viewport looks at **JavaScript errors**: a page can have not a single one and still run a hundred and sixty-three pixels off the screen. It was a requirement declared covered and not measured. The bench measures the document's `scrollWidth` — exactly what makes the scrollbar appear — and not any element wider than the screen, which inside a container built to scroll is correct and would fill the output with noise until nobody reads it any more.

What the accessibility check does not cover, and it must be said: text inside SVGs (the colour comes from `fill`, the background is a drawn shape) and text on gradient backgrounds (they do not have *a* colour). The skipped elements are counted and reported, not hidden.

One section at a time. The inspection skips elements that are not visible — and rightly so: a hidden element has no contrast to measure. But `index.html` alternates 96 sections and shows only one: out of **19,751** text elements the bench was looking at **41**, and printing "0 defects". It was not a false result, it was a result on a sample nobody had declared. The sections are now opened one by one, and **the number of sections traversed is printed**.

That count earned its keep on the very first run: the first attempt traversed **zero** — Playwright's click waits for the element to be visible, and the buttons sit inside collapsed navigation groups — and the bench said so ("87 sections present, none traversed") instead of printing another reassuring zero.

3.5-bis The accessibility debt, and the pact that it must not grow

Walking every section, **1,069** contrast defects surfaced. Today they are **zero** — the list of causes and remedies is in `docs/09` §4 — and unlabelled fields have fallen from 40 to **zero**.

The mechanism described below remains, and matters more than the number it guards. Demanding zero *from the outset* would have left only two roads, both bad: a verification red for ever, or one loosened until it turned green again. The third road is to **declare the number** and forbid it from growing, whatever it is. That is how 1,069 became zero: every step recorded, none of them reversible in silence.

File	Role
`docs/evidence/accessibilita-riferimento.json`	The recorded count

Situation	Bench outcome
The count **rises**	✗ FAILED — it is a regression
The count **falls**	✓ passes, and asks for the baseline to be updated
The count is **unchanged**	✓ passes silently: debt declared, not grown — today that value is **0**, and the bench defends it

```
node tools/verifica-accessibilita.js --aggiorna-riferimento
```

To be used **only after reducing** the defects, never to turn a regression green. Even at zero this is not full WCAG conformance: text inside SVGs and text over gradients stay outside, which automation cannot judge and which the bench **counts and reports** on every run (`docs/09` D-09). It is the honest measure of what can be measured, with the guarantee that it will not silently get worse.

3.6 Consistency

Bench	Verifies
`verifica_guida`	That the documentation's promises exist in the code; absence of conflict markers across 58 files
`verifica-documenti`	Internal links, version alignment, index consistency, justification of the deviations, **internal consistency of the traceability matrix** (no duplicated identifiers, coverage totals equal to the identifiers actually defined)
`genera-pacchetti`	Rewrites the references of the extracted documents and **verifies that no link is left broken** in the three delivery packages. It also verifies that **no PDF is older than the document it comes from** and that the bundled verification report declares **this** version: a typeset document or an evidence report from the previous version, inside a current package, is a silent lie
`genera-pdf`	Converts the documents to PDF and fails if a file falls below the plausibility threshold (empty conversion)
`verifica-affermazioni`	Compares every number declared in the documents — sections, drugs, diseases, tumours, strategies, modules — with the one **measured in the running application**, in Italian *and* in English

Bench	Verifies
`test_stack`	That the React/FastAPI stack runs **the same files** as the application: it compares the SHA-256 fingerprints `GET /salute` declares with those of the files on disk, checks the numbers match the browser's, and that **no `.ts` / `.tsx` file contains a shift table** — because that is where the second engine would be born

> **Why numbers must be measured, not remembered.** The technical dossier opens with figures: "63 diseases (23 tumours)", "46 strategies", "25 modules". They are the first thing an assessor reads, and they are enough for everything else to be believed or doubted. Nobody was checking them: written once, they stayed put while the application changed. And indeed `chimorga.html` declared **"25 modules"** at the top and **"17 modules"** further down — two statements about the same object, in the same file, one of them false.
>
> **A lesson paid for while writing the bench.** I had already concluded that "63 diseases" was an exaggeration: in the source I had found a `DISEASES` array with 12 entries, and I was about to "correct" the document. Measuring in the DOM I saw that the menu really does list 63 — the array I had found was a different, smaller one. **A count on the source is not a measurement: it is a clue.** That is why the bench opens the page, opens the section, and counts.
>
> It also checks that the documents agree **with one another**: two documents saying different things about the same object are a defect even when one of them is right. The English translation counts as one of those documents — `docs/en/05` had been left at "84 sections" while the app had 87, and had been stuck at version `bsi-v146` for three releases — and no bench saw it, because none looked at the English set.

> **Why a check on the matrix.** The matrix declared `SCI-10` twice — once as a requirement verified at 100 %, once as an explicitly **unverified** one — and the summary table added up to 36 requirements while 37 were defined in the document. Both errors sat in the number an assessor reads first. The bench now counts the identifiers actually present and fails if the totals written do not match: the proof that the check really measures something is that, with the two errors reintroduced one at a time, it reports both.

---

4. Notable test cases

Some benches deserve a mention because they verify properties a functional test does not observe.

Case	Method	Measured result
Memory leaks	96 sections × 5 rounds, DOM nodes counted each round	+26,876 on the first round (construction), **+0** on the four that follow
Exhausted storage	`setItem` replaced with a function that always throws	10 pages out of 10 stay operational

Case	Method	Measured result
Degraded ≠ absent network	Requests held for 20 s, interception at context level	Answer from cache in **3,507 ms** (threshold 3,500)
Spectrum reproducibility	Drawn twice, compared byte for byte	Identical
WebGL contexts	Viewer constructions over 10 consecutive molecules	From **7 to 1**
Corrupted history	Two `Enter` presses 500 ms apart	`user,assistant` instead of `user,user,assistant,assistant`
Text contrast	WCAG formula on every element with text of its own, 13 pages **and 96 sections**	19,751 elements examined (it was 41): **1,069** defects surfaced, **1,069 corrected**, **0 recorded** as a value that must not grow
Immediate cancellation	Stop pressed at 1.5 s, state sampled every second	Send button available from the **1st** second (it was the 10th)

5. How to add a bench

Minimal structure

```
/* A comment explaining WHAT this bench demonstrates and why it is needed. */
const { chromium } = require('playwright-core');
const CHROME = '/opt/pw-browsers/chromium-1194/chrome-linux/chrome';

(async () => {
  const b = await chromium.launch({ executablePath: CHROME, args: ['--no-sandbox'] });
  // serviceWorkers:'block' if the bench modifies files: without it the SW
  // might serve a cached copy and the old file would be measured
  const ctx = await b.newContext({ serviceWorkers: 'block' });
  const pg = await ctx.newPage();
  const err = [];
  pg.on('pageerror', e => err.push(e.message));

  let ok = 0, ko = 0;
  const att = (d, atteso, avuto) => {
    if (String(atteso) === String(avuto)) { ok++; console.log('  ✓ ' + d + '  → ' + avuto); }
    else { ko++; console.log('  ✗ ' + d + '\n      expected: ' + atteso + '\n      got:      '
+ avuto); }
  };

  await pg.goto('http://127.0.0.1:8899/index.html', { waitUntil: 'load', timeout: 60000 });
  await pg.waitForTimeout(2500);

  // … measurements …

  att('no JS error', 0, err.length);
  await b.close();
  console.log('\n' + (ko ? '✗ ' + ko + ' FAILED, ' : '') + ok + ' passed');
  process.exit(ko ? 1 : 0);   // the exit code is what counts
})();
```

The name is then added to the appropriate family in `tools/genera-evidenza.js`.

The rule that matters

> **A bench that measures nothing passes.** It is the most insidious way to obtain a false guarantee: no alarm, no failure, and no verification.

Therefore **every bench that simulates a condition must count how many times the simulation actually fired, and fail if that count is zero.**

Traps actually met while building the battery:

Trap	How it showed itself
`page.route` does not intercept Service Worker requests (`context.route` is needed)	The "degraded network" test measured a healthy network and concluded in 16 ms instead of 3,500
Probing exhausted `localStorage` with a one-byte write	After saturation one byte always finds room: the bench "passed"
Looking for `[data-tab]` where navigation uses `[data-s]`	Zero tabs found, therefore zero measurements — and zero failures
The ticks of an axis on `<canvas>` are not text	They must be collected by intercepting `fillText`; and the bottom-most line is the axis *title*, not a tick
A probe installed on a library loaded **afterwards**	It is replaced along with the library: zero calls counted while the code was working
A `var` inside an IIFE is not `window.something`	Assigning it from outside creates a different variable

---

6. Coverage gaps

Declared. The coverage reported in `08-Traceability-Matrix.md` measures **how many requirements have a bench**, not how many lines of code are executed: they are two different things and must not be confused.

Gap	Current situation	Recommendation
Code coverage	Measured: **49.89 %** of statements over the widest path. What stays outside is whole-battery coverage and **branch** coverage: an `if` entered from one side only counts as covered	Extend collection to every bench, and move from statement coverage to branch coverage
Accessibility	Automated over 13 pages and all 96 sections: WCAG contrast, accessible names, labels, alternative text, heading hierarchy. Text inside SVGs and over gradients stay outside, and are **counted** on every run	Add `axe-core` alongside, for the rules this bench does not implement (ARIA roles, tab order, focus management)
Security	`verifica-sicurezza` runs 10 checks over 372 tracked files and covers SEC-01, SEC-03, SEC-05, SEC-06; SEC-04 is covered by `verifica_guida`. **SEC-02 remains indirect**: see `docs/09` D-03	Observe the network traffic during real use, the only direct verification of SEC-02

Gap	Current situation	Recommendation
Browsers other than Chromium	No automatic test on Firefox or WebKit	Extend the main benches to `webkit`, where the differences on IndexedDB and Service Worker are greatest
Performance	Manual cross-device testing	Automatic measurement of first-paint time
Visual regression	Absent	Screenshot comparison for the charts, which are the heart of the product

---

7. Acceptance criteria

A version is not published if even one of these is unsatisfied.

- ☐ `node tools/genera-evidenza.js` closes with **0 failures**
- ☐ `tools/verifica-farmaci.js`: no defect, every deviation recorded with its reason
- ☐ `verifica_guida`: no promise in the documentation without a counterpart
- ☐ No non-network JavaScript error on any page
- ☐ Evidence report regenerated against the commit being published

---

Document updated to version `bsi-v203`.